

Memory Management

Chapter 7

Agenda

- Discuss the principal requirements for memory management.
- Understand the reason for memory partitioning and explain the various techniques that are used.
- Understand and explain the concept of paging.
- Understand and explain the concept of segmentation.
- Assess the relative advantages of paging and segmentation.
- Summarize key security issues related to memory management.
- Describe the concepts of loading and linking.

Memory Management

- What???
 - Uniprogramming System Vs Multiprogramming System
- Subdividing **user part of** memory to accommodate multiple processes dynamically.
- Why???
- Memory needs to be allocated to ensure a reasonable supply of ready processes to consume available processor time

Memory Management Requirements

- Purpose????
 - The requirements that memory management is intended to satisfy
- Relocation
 - Programmer does not know where the program will be placed in memory when it is executed
 - While the program is executing, it may be swapped to disk and returned to main memory at a different location (relocated)
 - Memory references must be translated in the code to actual physical memory address

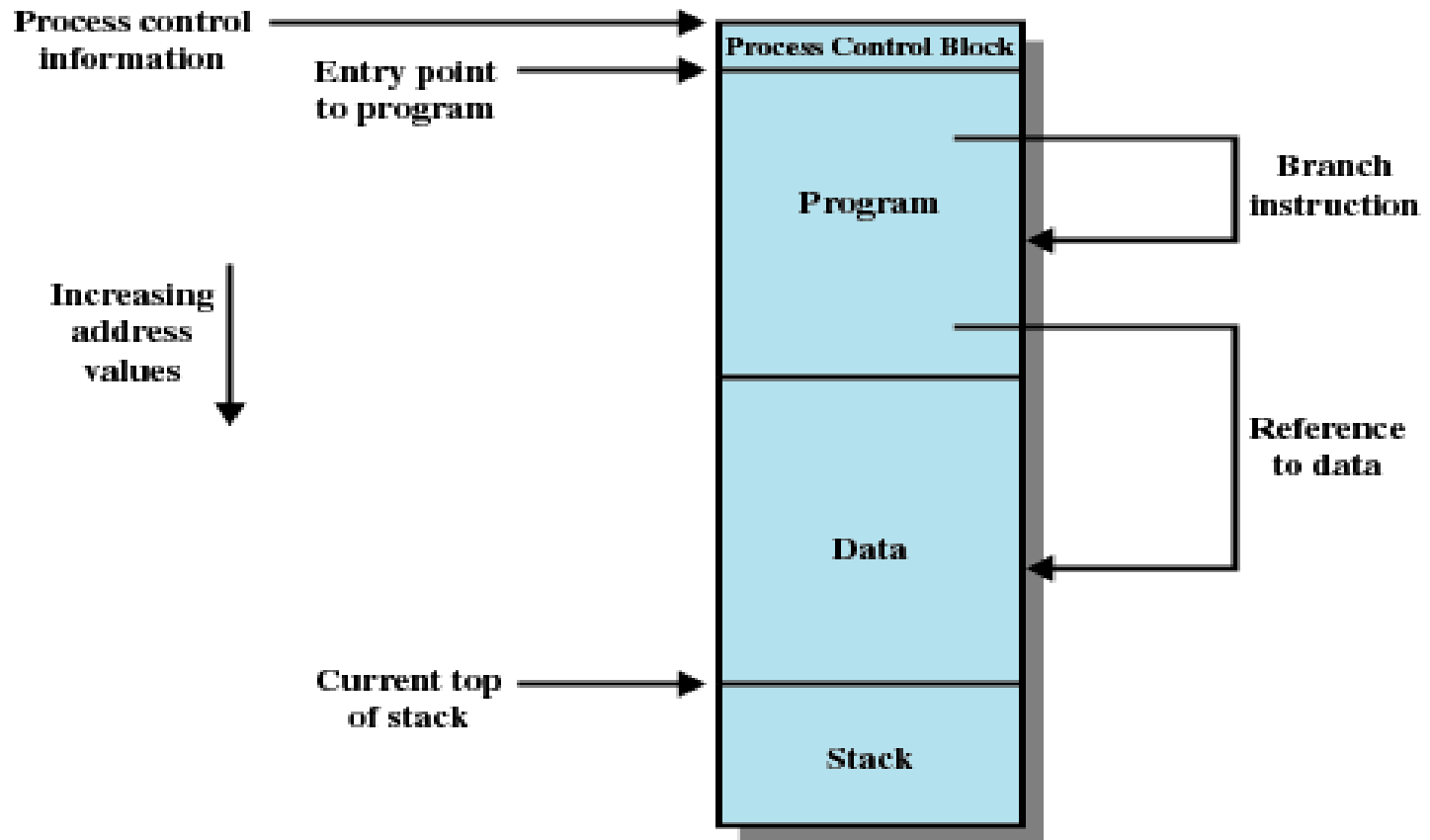


Figure 7.1 Addressing Requirements for a Process

Memory Management Requirements

- Protection
 - Processes should not be able to reference memory locations in another process without permission
 - Impossible to check absolute addresses at compile time
 - Must be checked at run time
 - Memory protection requirement must be satisfied by the processor (hardware) rather than the operating system (software)
 - Operating system cannot anticipate all of the memory references a program will make

Memory Management Requirements

- Sharing
 - Allow several processes to access the same portion of memory
 - To allow each process access to the same copy of the program rather than have their own separate copy

Memory Management Requirements

- Logical Organization
 - main memory in a computer system is organized as a linear 1D address space consisting of a sequence of bytes or words.
 - Programs are written in modules
 - Modules can be written and compiled independently
 - Different degrees of protection given to modules (read-only, execute-only)
 - Share modules among processes

Memory Management Requirements

- Physical Organization
 - Memory available for a program plus its data may be insufficient
 - Overlaying allows various modules to be assigned the same region of memory
 - Programmer does not know how much space will be available
 - **The task of moving information between the two levels of memory should be a system responsibility**

Memory Management Requirements

Essence of memory management.

- The task of moving information between the two levels of memory should be a **system responsibility Not Programmer's**

Memory Management Requirements

Principle responsibility of memory management

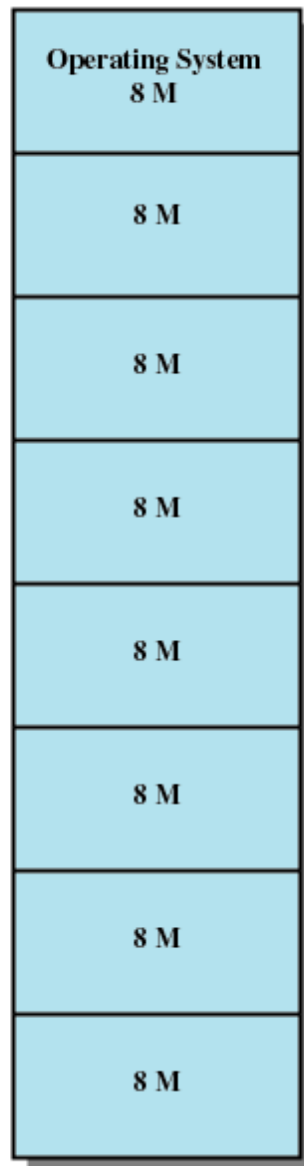
- To bring processes into main memory for execution by the processor to ensure the sufficient no. of ready processes.

Memory Partitioning: Fixed Partitioning

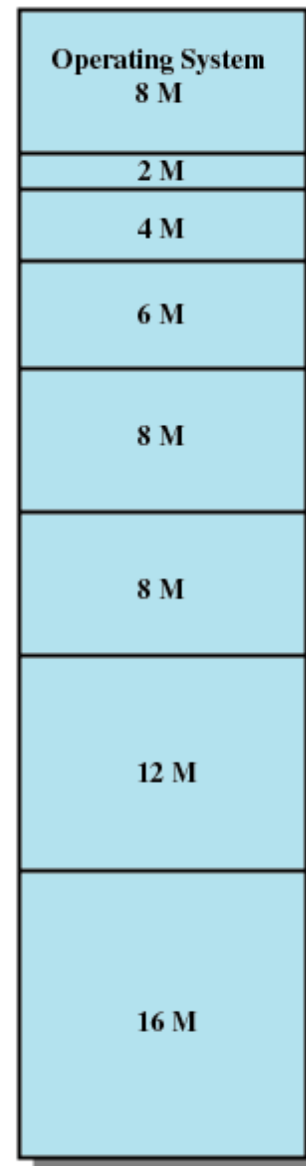
- Equal-size partitions
 - Any process whose size is less than or equal to the partition size can be loaded into an available partition
 - If all partitions are full, the operating system can swap a process out of a partition
 - A program may not fit in a partition. The programmer must design the program with overlays

Fixed Partitioning

- Main memory use is inefficient. Any program, no matter how small, occupies an entire partition. This is called **internal fragmentation**.
- A program may not fit in a partition. It become programmers headache to write the program in overlayed form.



(a) Equal-size partitions

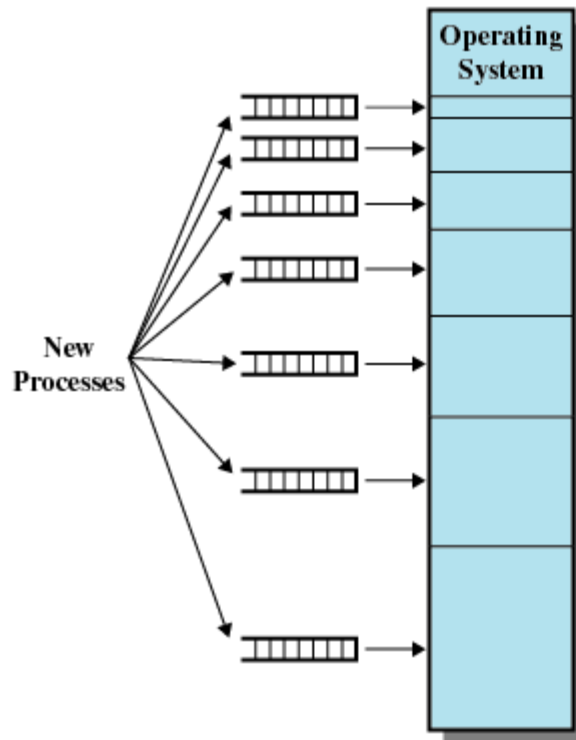


(b) Unequal-size partitions

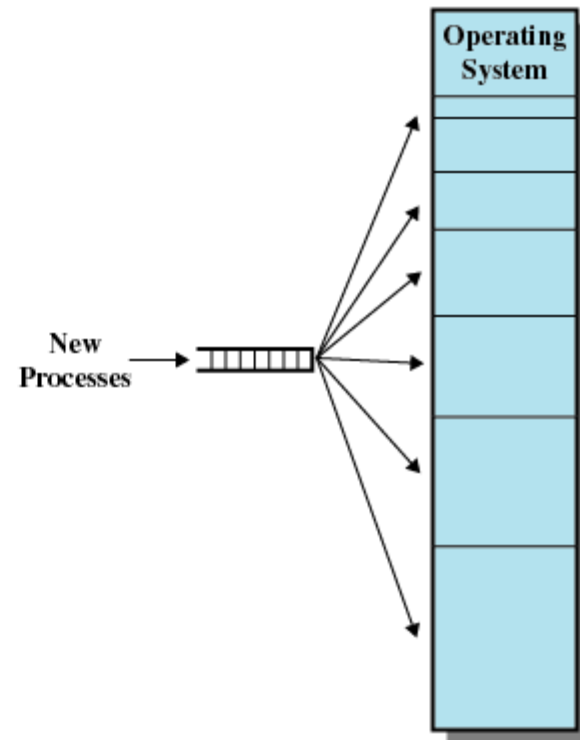
Figure 7.2 Example of Fixed Partitioning of a 64-Mbyte Memory

Placement Algorithm with Partitions

- Equal-size partitions
 - Because all partitions are of equal size, it does not matter which partition is used
- Unequal-size partitions
 - Can assign each process to the smallest partition within which it will fit
 - Queue for each partition
 - Processes are assigned in such a way as to minimize wasted memory within a partition



(a) One process queue per partition



(b) Single queue

Figure 7.3 Memory Assignment for Fixed Partitioning

Dynamic Partitioning

- Partitions are of variable length and number
- Process is allocated exactly as much memory as required
- Eventually get holes in the memory. This is called external fragmentation
- Must use compaction to shift processes so they are contiguous and all free memory is in one block

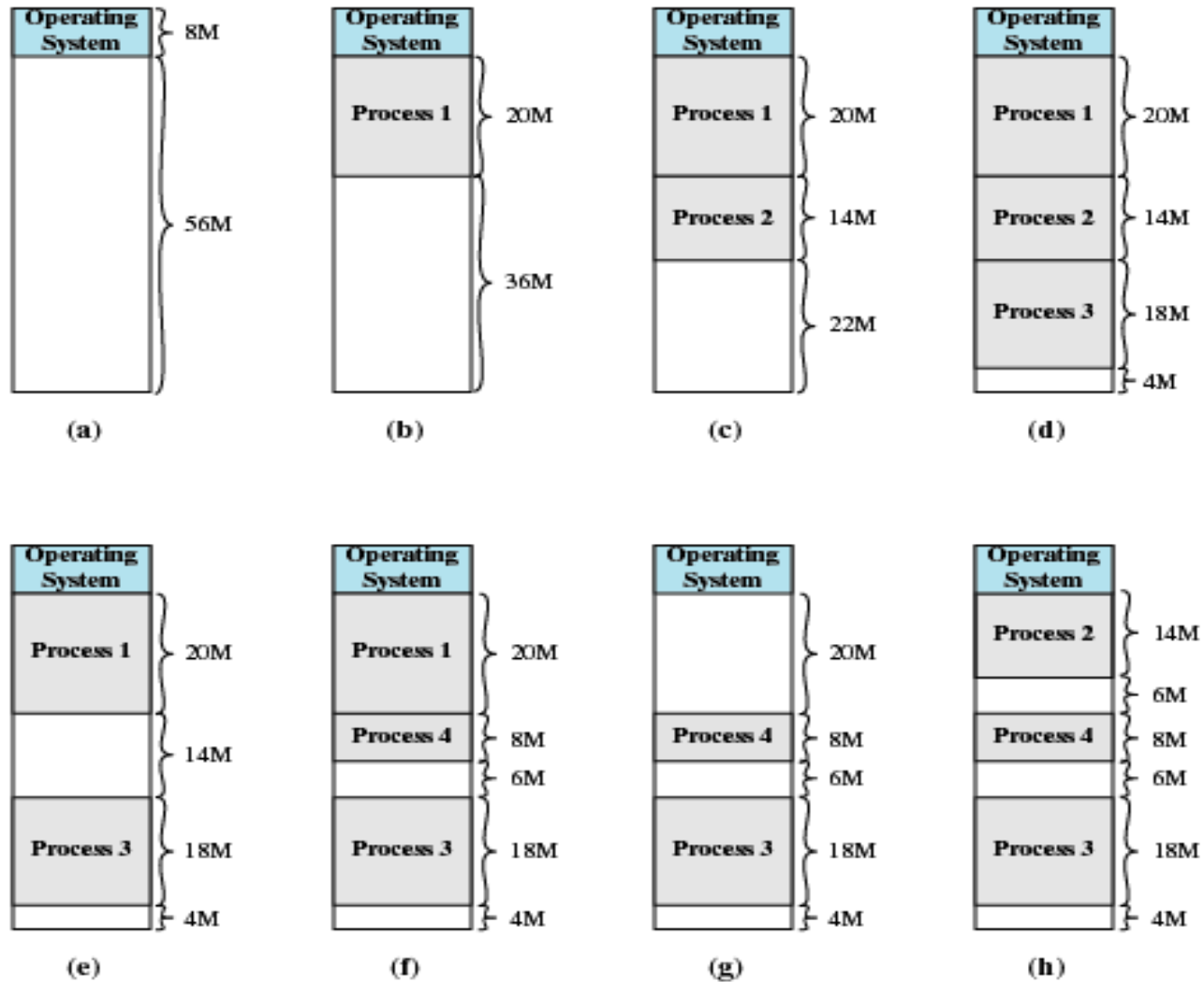


Figure 7.4 The Effect of Dynamic Partitioning

Dynamic Partitioning Placement Algorithm

- Operating system must decide which free block to allocate to a process
- Best-fit algorithm
 - Chooses the block that is closest in size to the request
 - Worst performer overall
 - Since smallest block is found for process, the smallest amount of fragmentation is left
 - Memory compaction must be done more often

Dynamic Partitioning Placement Algorithm

- First-fit algorithm
 - Scans memory from the beginning and chooses the first available block that is large enough
 - Fastest
 - May have many process loaded in the front end of memory that must be searched over when trying to find a free block

Dynamic Partitioning Placement Algorithm

- Next-fit
 - Scans memory from the location of the last placement
 - More often allocate a block of memory at the end of memory where the largest block is found
 - The largest block of memory is broken up into smaller blocks
 - Compaction is required to obtain a large block at the end of memory

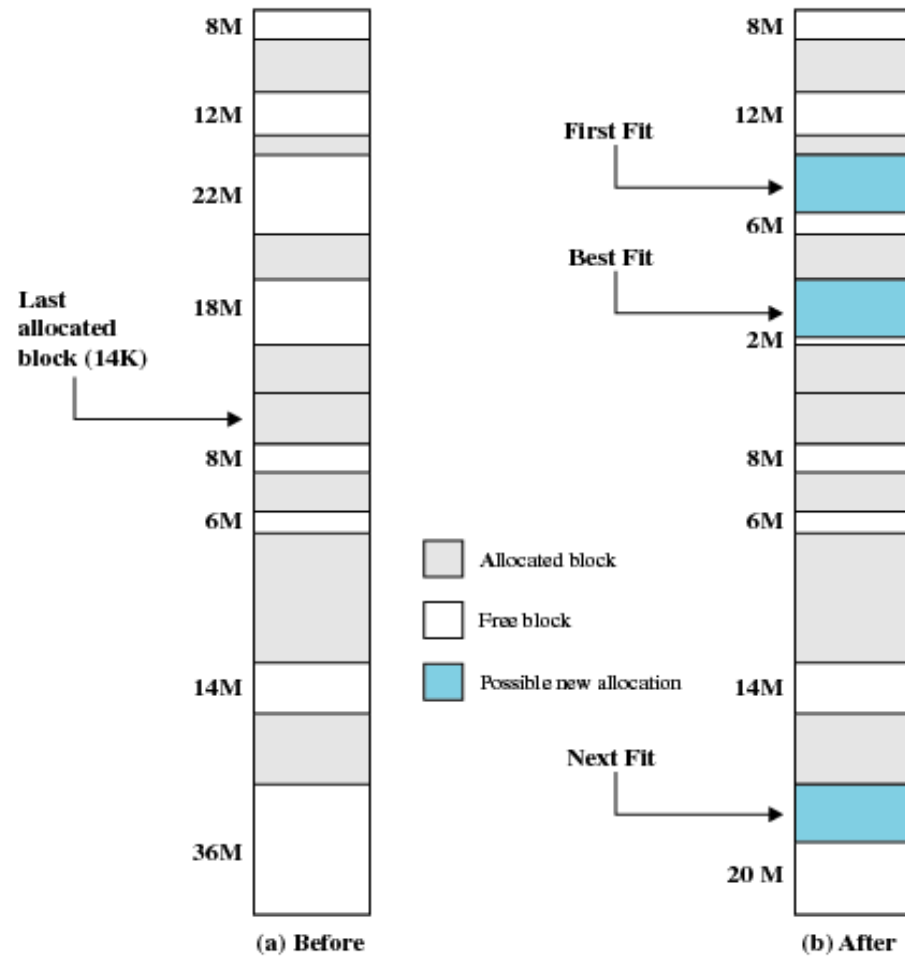


Figure 7.5 Example Memory Configuration Before and After Allocation of 16 Mbyte Block

Buddy System

- Both fixed and dynamic partitioning schemes have drawbacks.
 - A fixed partitioning scheme **limits the number of active processes**
 - May use space inefficiently if there is a **poor match between available partition sizes and process sizes.**
 - A dynamic partitioning scheme is more **complex to maintain**
 - Also includes the **overhead of compaction.**

Buddy System

- Memory blocks are available of size 2^K words, $L \leq K \leq U$
- 2^L **smallest size block** that is allocated
- 2^U **largest size block** that is allocated; generally 2^U is the size of the entire memory available for allocation.
- **Initially,**
- Entire space available is treated as a single block of 2^U
- If a request of size **s** such that $2^{U-1} < s \leq 2^U$, entire block is allocated
 - Otherwise block is split into two equal buddies
 - Process continues until smallest block greater than or equal to **s** is generated.

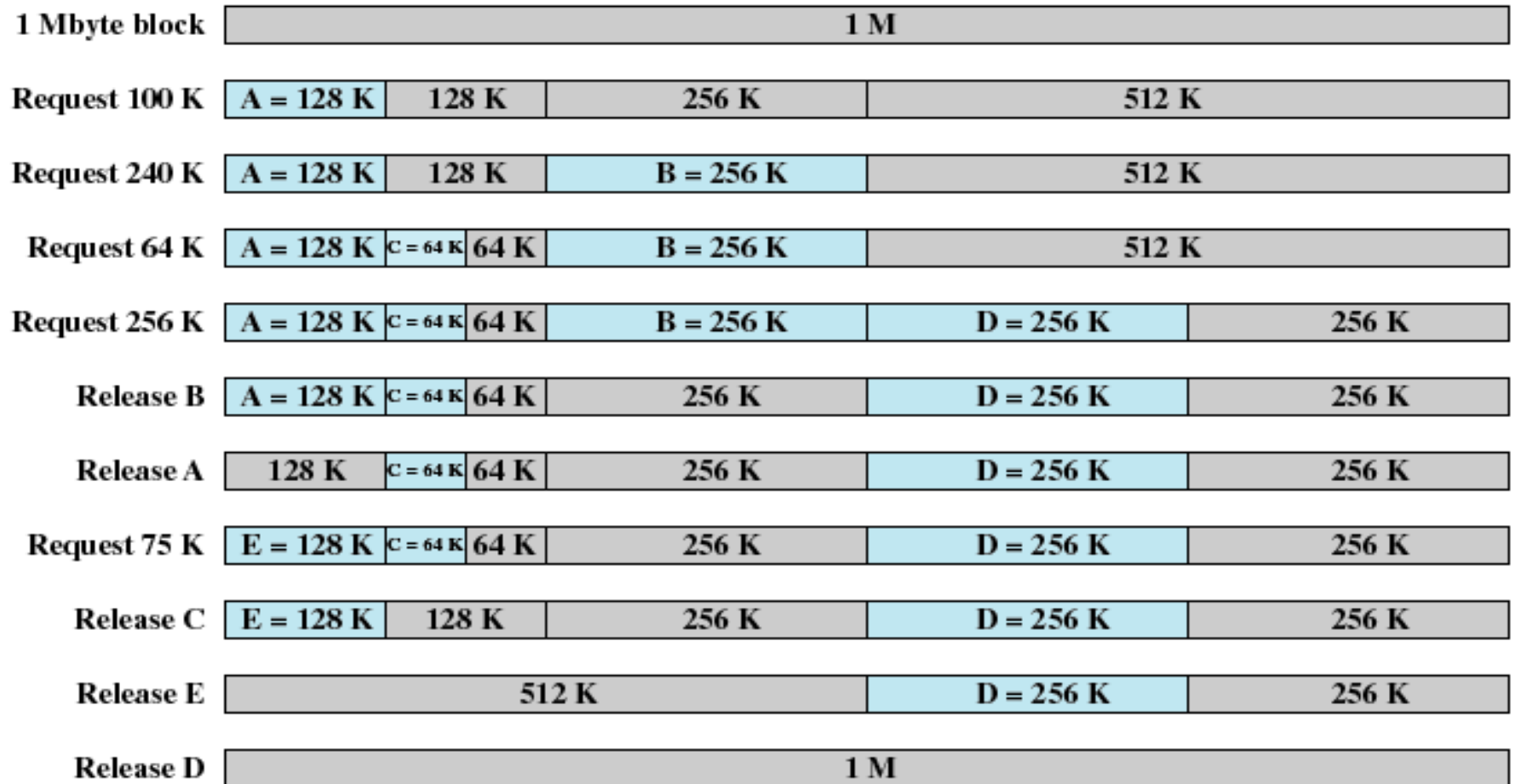


Figure 7.6 Example of Buddy System

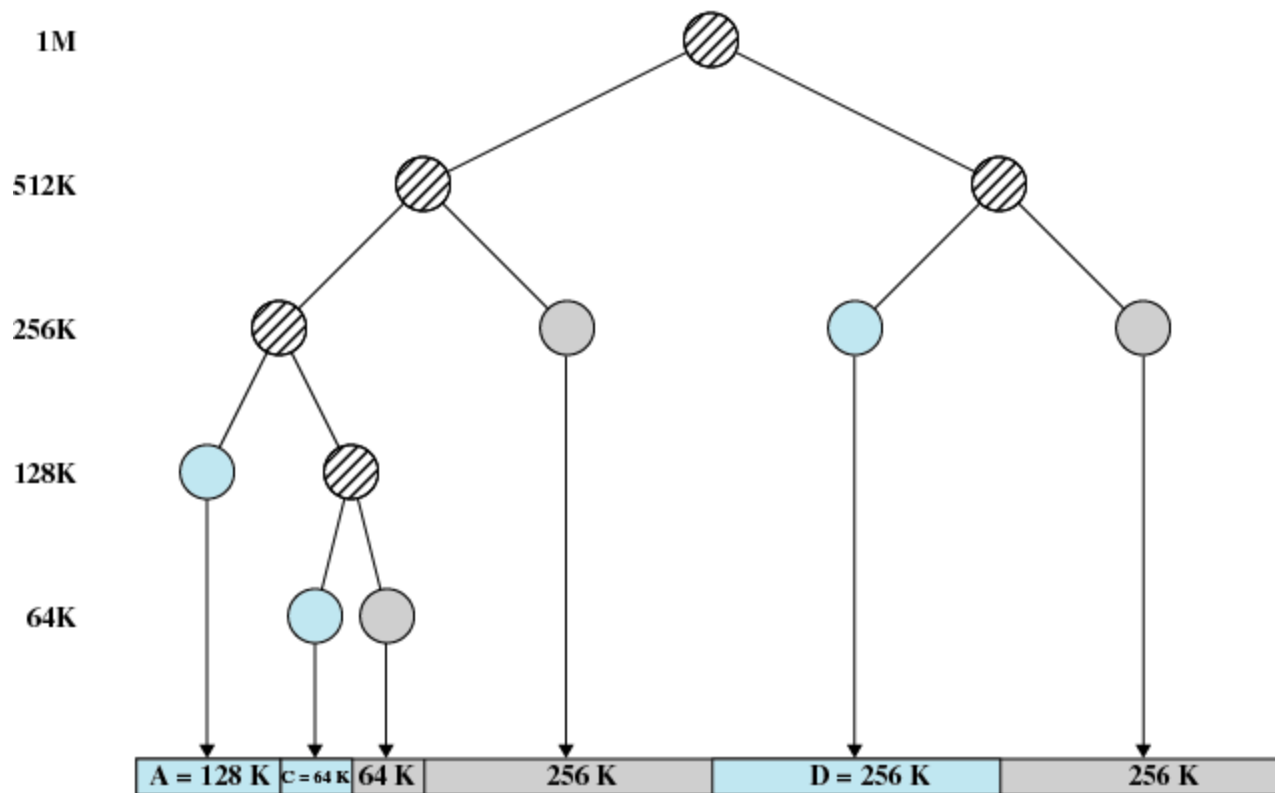


Figure 7.7 Tree Representation of Buddy System

Buddy Systems

- Application of Buddy systems:
 - In parallel systems as an efficient means of allocation and release for parallel programs.
- A modified form of the buddy system is used for **UNIX kernel memory allocation**

What next????

- Ways of dealing with the shortcomings of partitioning let it be any
 - **Fixed**
 - **Equal sized**
 - **Unequal sized**
 - **Dynamic**
 - **Buddy**

What next????

- A process may occupy different partitions during the course of its life.
- When a process image is first created, it is loaded into some partition in main memory.
- The process may be swapped out;
- When it is subsequently swapped back in, it may be assigned to a different partition than the last time.

Relocation

- When program loaded into memory the actual (absolute) memory locations are determined.
- A process may occupy different partitions which means different absolute memory locations during execution (from swapping)
- Compaction will also cause a program to occupy a different partition which means different absolute memory locations.

Addresses

- A distinction is made among several types of addresses.
- Logical
 - Reference to a memory location **independent** of the current assignment of data to memory
 - Translation must be made to the **physical address**
- Relative
 - Address expressed as a location relative to some known point (Type of logical address)
- Physical
 - The absolute address or actual location in main memory

Registers Used during Execution

- Base register
 - Starting address for the process
- Bounds register
 - Ending location of the process
- These values are set when the process is loaded/updated or when the process is swapped in.

Typically, all of the memory references in the loaded process are relative to the origin of the program

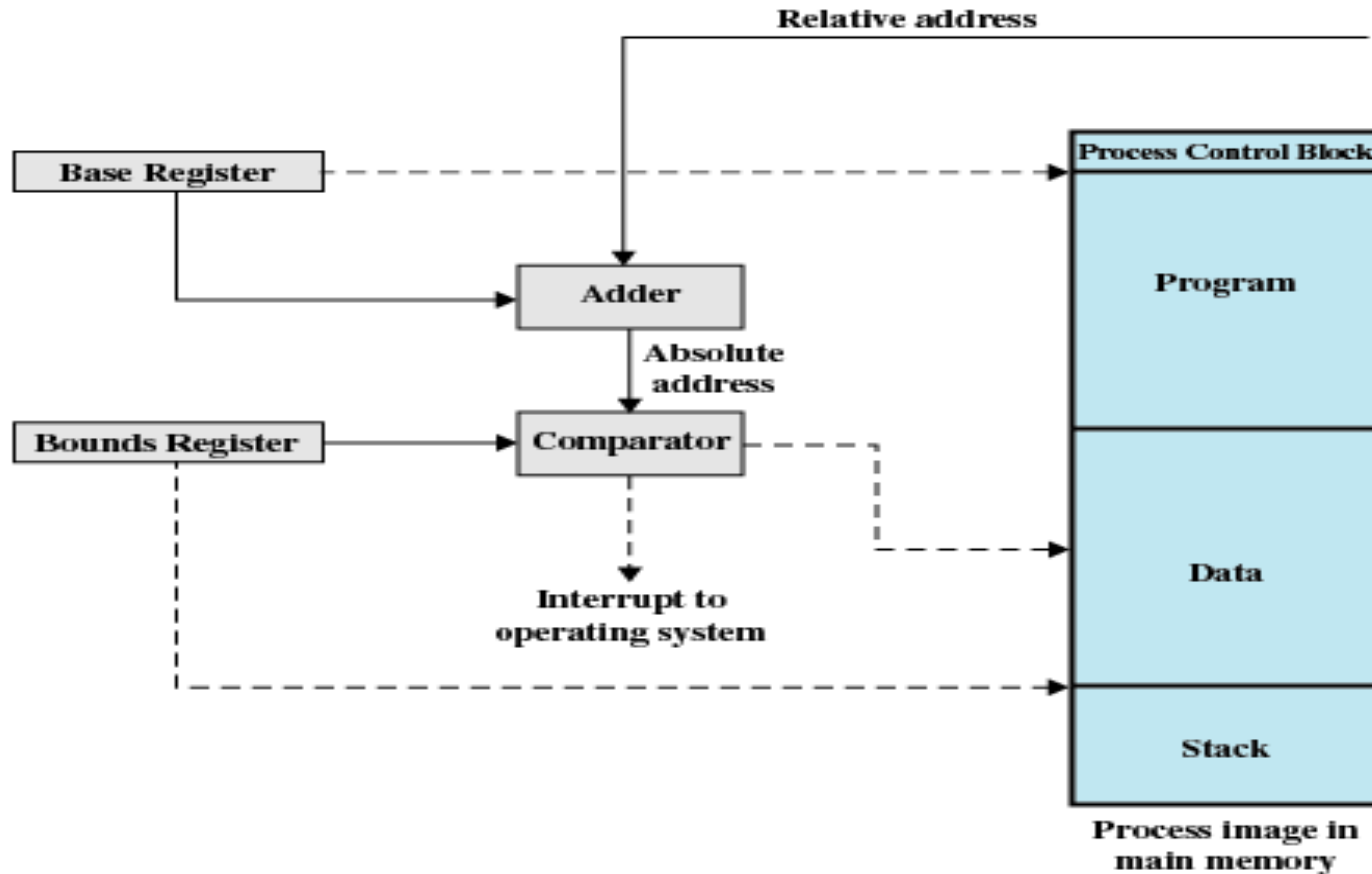


Figure 7.8 Hardware Support for Relocation

Process of Relocation

- The value of the **base register** is **added** to a **relative address** to **produce** an **absolute address**.
- The **resulting address** is **compared** with the value in the **bounds register**.
- If the address is **not within** bounds, an **interrupt** is generated to the operating system

Small exercise

- Consider a fixed partitioning scheme with equal-size partitions of 2^{16} bytes and a total main memory size of 2^{24} bytes. A process table is maintained that includes a pointer to a partition for each resident process. How many bits are required for the pointer?

Paging:

- Partition memory into small **equal fixed-size** chunks and divide each process into the same size chunks
- The **chunks of a process** are called **pages** and **chunks of memory** are called **frames**
- Operating system maintains **a page table** for **each process**
 - Contains the frame location for each page in the process
 - Memory address consist of a page number and offset within the page

Assignment of Process Pages to Free Frames

Frame number	Main memory
0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
14	

(a) Fifteen Available Frames

Frame number	Main memory
0	A.0
1	A.1
2	A.2
3	A.3
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
14	

(b) Load Process A

Frame number	Main memory
0	A.0
1	A.1
2	A.2
3	A.3
4	B.0
5	B.1
6	B.2
7	
8	
9	
10	
11	
12	
13	
14	

(c) Load Process B

Assignment of Process Pages to Free Frames

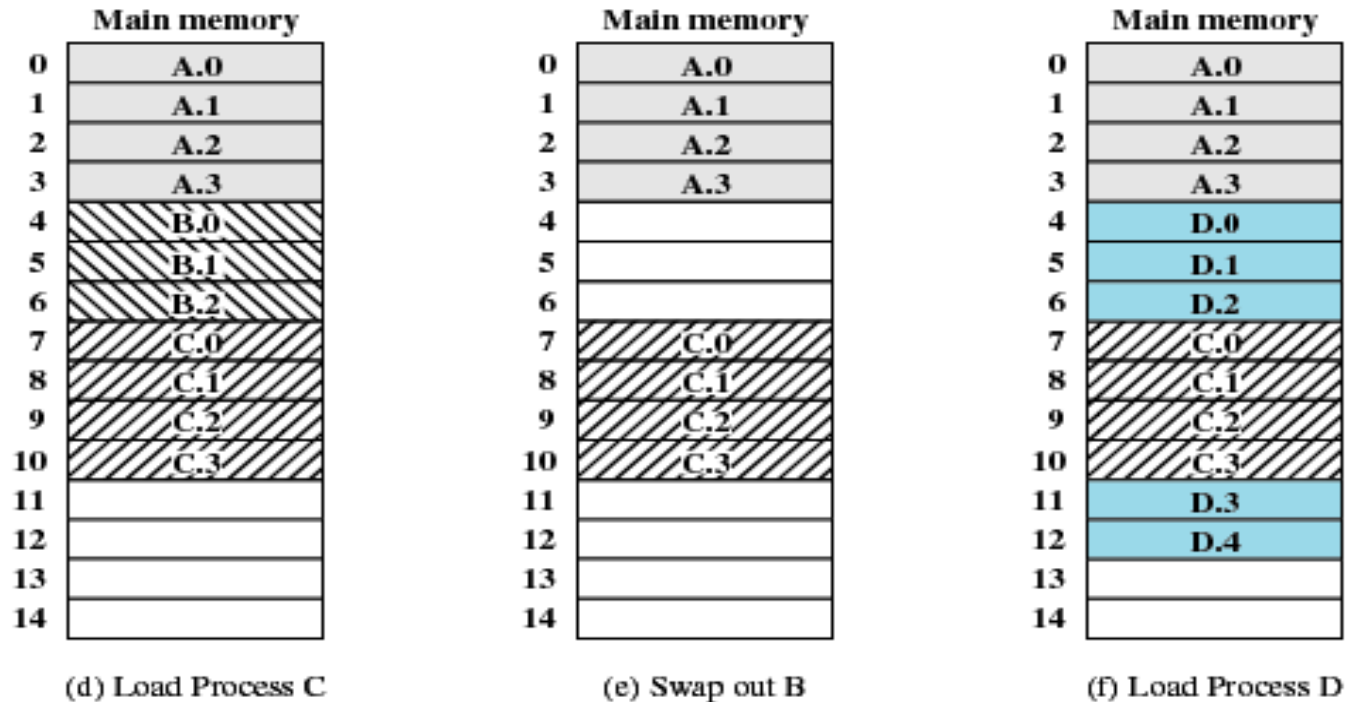


Figure 7.9 Assignment of Process Pages to Free Frames

Page Tables for Example

0	0
1	1
2	2
3	3

Process A
page table

0	N
1	N
2	N

Process B
page table

0	7
1	8
2	9
3	10

Process C
page table

0	4
1	5
2	6
3	11
4	12

Process D
page table

13
14

Free frame
list

Figure 7.10 Data Structures for the Example of Figure 7.9 at Time Epoch (f)

Segmentation

- All segments of all programs do not have to be of the same length
- There is a maximum segment length
- Addressing consist of two parts - a segment number and an offset
- Since segments are not equal, segmentation is similar to dynamic partitioning

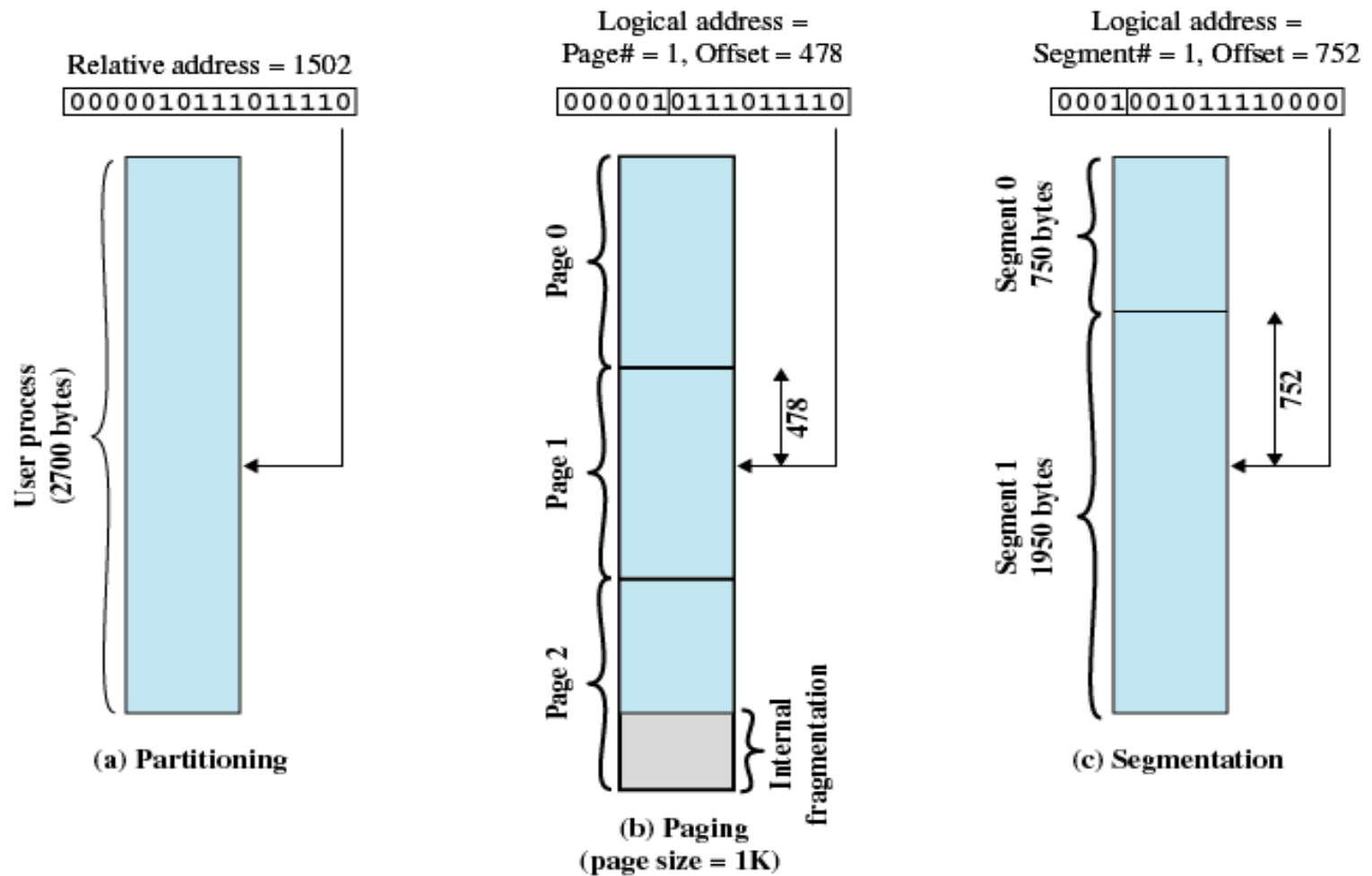
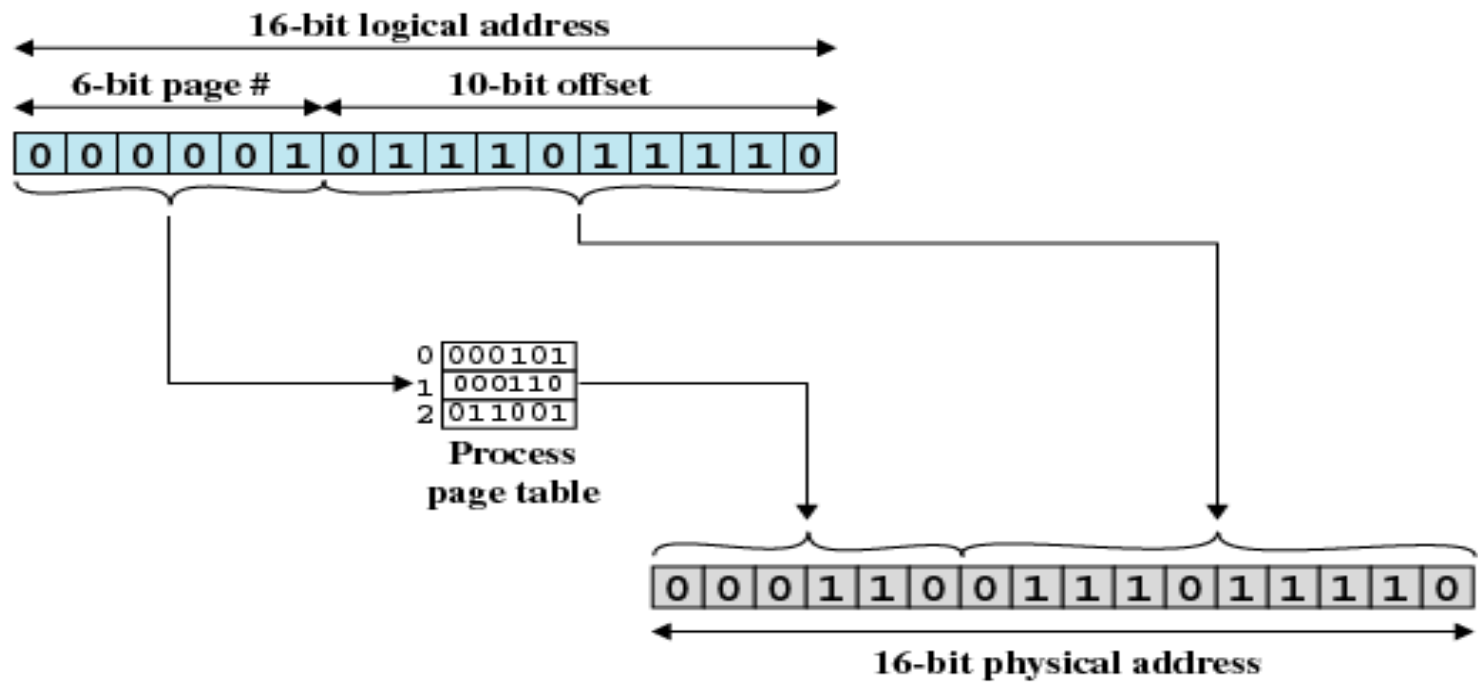


Figure 7.11 Logical Addresses



(a) Paging

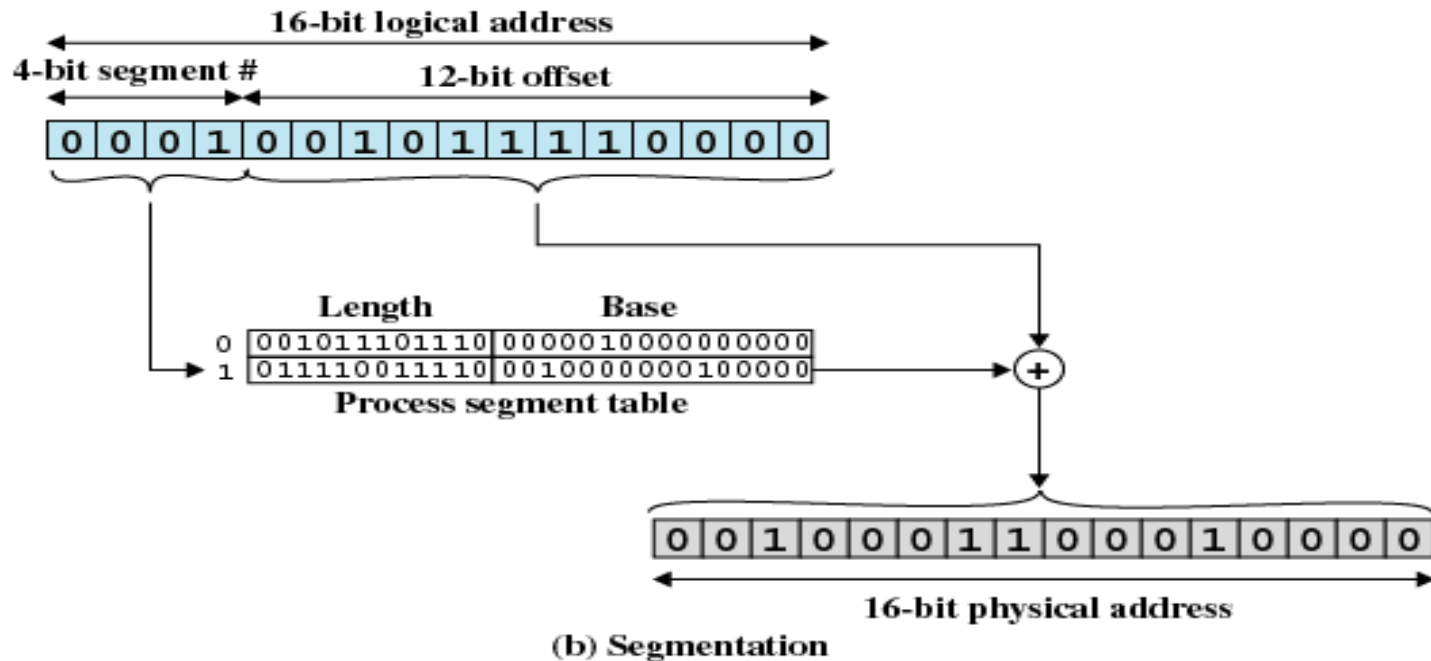


Figure 7.12 Examples of Logical-to-Physical Address Translation

Technique	Description	Strengths	Weaknesses
Fixed Partitioning	Main memory is divided into a number of static partitions at system generation time. A process may be loaded into a partition of equal or greater size.	Simple to implement; little operating system overhead.	Inefficient use of memory due to internal fragmentation; maximum number of active processes is fixed.
Dynamic Partitioning	Partitions are created dynamically, so that each process is loaded into a partition of exactly the same size as that process.	No internal fragmentation; more efficient use of main memory.	Inefficient use of processor due to the need for compaction to counter external fragmentation.
Simple Paging	Main memory is divided into a number of equal-size frames. Each process is divided into a number of equal-size pages of the same length as frames. A process is loaded by loading all of its pages into available, not necessarily contiguous, frames.	No external fragmentation.	A small amount of internal fragmentation.
Simple Segmentation	Each process is divided into a number of segments. A process is loaded by loading all of its segments into dynamic partitions that need not be contiguous.	No internal fragmentation; improved memory utilization and reduced overhead compared to dynamic partitioning.	External fragmentation.
Virtual Memory Paging	As with simple paging, except that it is not necessary to load all of the pages of a process. Nonresident pages that are needed are brought in later automatically.	No external fragmentation; higher degree of multiprogramming; large virtual address space.	Overhead of complex memory management.
Virtual Memory Segmentation	As with simple segmentation, except that it is not necessary to load all of the segments of a process. Nonresident segments that are needed are brought in later automatically.	No internal fragmentation, higher degree of multiprogramming; large virtual address space; protection and sharing support.	Overhead of complex memory management.

Let us do :

A 1-Mbyte block of memory is allocated using the buddy system. a. Show the results of the following sequence in a figure

Request 70; Request 35; Request 80; Return A; Request 60; Return B; Return D; Return C. b. Show the binary tree representation following Return B.

Let us do : (For chapter 8)

Consider a simple paging system with the following parameters: 2^{32} bytes of physical memory; page size of 2^{10} bytes; 2^{16} pages of logical address space.

- a. How many bits are in a logical address?
- b. How many bytes in a frame?
- c. How many bits in the physical address specify the frame?
- d. How many entries in the page table?
- e. How many bits in each page table entry?

- Physical: 2^{32}
- Page Size: 2^{10}
- Number of Pages: 2^{16}
 - **How many bits in a logical address?** That's the page address bits plus the number of pages bits. The upper portion of an address is the page number (16 bits), and the lower portion is the offset within that address (10 bits), so the whole address size is 26 bits (2^{26} bytes).

- Physical: 2^{32}
- Page Size: 2^{10}
 - Number of Pages: 2^{16} **How many bytes are in a frame?** A frame is where a page can be mapped into memory, so a frame has to be the same size as a page - 2^{10} bytes.
 - **How many bits are in the physical address specifying the frame?** Well, you have 32 bits of physical address, and a frame is 2^{10} big, so that leaves 22 of the bits ($32 - 10$) for the frame's base address.

- Physical: 2^{32}
- Page Size: 2^{10}
 - **How many entries in the page table?** The page table is the full list of pages, whether mapped or unmapped - so there are 2^{16} entries in the page table, since there are 2^{16} pages.

- Physical: 2^{32}
- Page Size: 2^{10}
 - **How many bits in each page table entry?**
Assume each page table entry contains a valid/invalid bit. If each page maps to an entry in the page table, and that table is a list of the addresses at which the pages are mapped in physical memory, then each address in the table must be the size of answer 3 (22 bits) plus one bit for the valid/invalid bit, so 23 bits.